

RELATÓRIO DE DIAGNÓSTICO E RESOLUÇÃO TÉCNICA

Tema: PIC-Reaproveitamento da água cinza de máquinas
de lavar roupa com válvula programável

Aluno: Artur Dornfeld de Almeida dos Reis

Curso/Semestre: Engenharia de Computação/5º

Data: 02/04/2026

1. Contexto

O presente relatório técnico documenta a contribuição feita para o PI, descrevendo o projeto antes das contribuições, necessidade de uma nova implementação, as opções de melhorias analisadas, justificativas da escolha final, resultado final e futura implementações.

2. Projeto Original

Anteriormente, o sistema operava em malha fechada, com a lógica de automação e controle da válvula restrita exclusivamente ao processamento local do microcontrolador.

Nessa arquitetura, o dispositivo não possuía interface de rede, o que impossibilitava o monitoramento remoto, o registro histórico de dados em nuvem ou qualquer interação externa com o sistema em tempo real. O funcionamento era isolado, dependendo apenas dos gatilhos físicos configurados no hardware.

3. Objetivo da contribuição

O objetivo central da contribuição é converter o sistema de controle local em uma plataforma de IoT (Internet das Coisas) integrada, fazendo-o funcional. Para viabilizar a demanda de monitoramento remoto, a contribuição técnica foca nos seguintes requisitos:

- Conectividade e Abstração: Implementar o uso de módulo Wifi no hardware(microcontrolador) e desenvolver uma camada de rede que permita ele se comunicar com o ambiente externo via protocolo de comunicação.
- Sincronismo em Tempo Real: Estabelecer uma infraestrutura de transmissão onde o estado físico do sistema (sensor/válvula) seja espelhado instantaneamente em um banco de dados em nuvem e conecte-se à aplicação.

4. Opções de melhorias analisadas

Para a transição da malha fechada para um sistema conectado, foram avaliadas diferentes combinações de hardware, conectividade e software. As principais alternativas consideradas foram:

4.1. Microcontrolador e Interface de Rede

Critério/	Arduino (Uno/Mega) + Módulo Externo	ESP32 (Escolha Final)
Processamento	Menor (8-bit, clock baixo); processamento linear que pode travar em tarefas de rede.	Superior (32-bit, Dual-Core); permite gerenciar a pilha de rede em um núcleo e o sensor no outro.
Conectividade	Não possui Wi-Fi nativo; exige módulos extras (ESP8266 ou Shields), aumentando a complexidade.	Wi-Fi e Bluetooth integrados no próprio chip, garantindo uma comunicação mais estável.
Dimensões	Maior ocupação de espaço físico no protótipo final.	Compacto; ideal para embutir em caixas de controle de válvulas pequenas.
Custo	Preço elevado ao somar a placa base com os módulos de rede necessários.	Baixo custo; solução "tudo-em-um" com melhor custo-benefício para o PIC.

4.2. Meio de Transmissão (Rede)

Critério	Ponto de Acesso Móvel (Celular)	Rede Fixa (Roteador Institucional/CEUB)
Estabilidade	Baixa: Sujeita a interferências do ambiente e oscilações de sinal do aparelho móvel.	Alta: Sinal constante e robusto, ideal para transmissões contínuas de telemetria.
Segurança da Rede	Simples: Proteção básica via WPA2, facilitando a conexão direta do microcontrolador.	Complexa: Pode exigir autenticação WPA2-Enterprise ou certificados que o hardware básico nem sempre suporta.
Consumo	Alto: Drena rapidamente a bateria e o plano de dados do smartphone pareado.	Irrelevante: Alimentação direta pela rede elétrica e tráfego de dados ilimitado.
Mobilidade	Alta: Permite testar o protótipo em diferentes pontos de coleta de água cinza.	Nula: O dispositivo fica restrito ao raio de alcance do roteador físico.
Largura de Banda	Limitada: Compartilhada com outros processos do celular, podendo causar latência.	Elevada: Capacidade de tráfego dedicada, garantindo a atualização instantânea do Firebase.

4.3. Protocolos e Integração com a Nuvem

Tecnologia / Método	Vantagens	Limitações e Desafios
Firestore Realtime Database (Planejado)	Sincronismo nativo e atualização por push (sem recarregar a página). Funcionamento excelente em redes sem firewall(residenciais).	Bloqueio de Firewall: Durante os testes em rede fixa segura(institucional do CEUB), o protocolo HTTPS (porta 443) sofreu bloqueios, impedindo a comunicação direta do ESP32 com o Firestore.
Webhooks HTTP (Contorno Técnico)	Permitiu contornar as restrições de segurança da rede institucional que bloqueavam conexões criptografadas complexas, mas ainda .	Custo: O uso de Webhooks simples via HTTP para serviços de terceiros (como Make.com) gerava dependência de créditos. Conexão com firestore: ainda em pesquisa.
Outras Integrações (Em Pesquisa)	Foram consideradas integrações com brokers MQTT ou APIs de terceiros para garantir redundância de dados	Inviabilidade Momentânea: Descartadas durante o desenvolvimento para priorizar o MVP.

4.4. Interface de usuário (Frontend/Backend)

Para a validação da camada de comunicação e entrega do MVP (Mínimo Produto Viável), a interface de usuário foi estruturada de forma simplificada e funcional. O foco desta etapa, sob minha responsabilidade, concentrou-se na integridade do fluxo de dados entre o microcontrolador e a nuvem, garantindo que o acionamento físico fosse registrado corretamente.

Dessa forma, foi desenvolvida uma página estática baseada em HTML5 e JavaScript, executada localmente (index.html), que se conecta diretamente à API do Firebase. Esta escolha técnica permitiu validar, em tempo real, se o estado do Pino 15 estava sendo transmitido sem perdas, utilizando a tecnologia de WebSockets para atualizar o status na tela sem a necessidade de recarregar o navegador.

Esta arquitetura foi adotada como uma solução de contorno eficiente, permitindo o progresso dos testes de hardware e rede de forma independente. Assim que as equipes de Frontend e Backend definirem a plataforma definitiva do aplicativo (seja em frameworks como React ou Node.js), a estrutura de comunicação via Google Apps Script e Firebase já estará consolidada, bastando apenas a migração das chaves de acesso para a nova interface.

5. Análise Técnica e Justificativa das Escolhas

A arquitetura final do projeto foi definida através de uma análise criteriosa entre as necessidades do sistema e as limitações de infraestrutura. As escolhas abaixo justificam a viabilidade do MVP (Mínimo Produto Viável):

- **Microcontrolador (ESP32):** A escolha do ESP32 em detrimento do Arduino baseou-se na superioridade de processamento (Dual-Core) e no Wi-Fi nativo. Isso eliminou custos com módulos externos e, principalmente, resolveu os erros de Watchdog e boot loops que ocorriam ao tentar processar a rede e a leitura do sensor (Pino 15) simultaneamente.
- **Conectividade (Rede Fixa):** Optou-se pela utilização da rede fixa em vez do Wi-Fi compartilhado de dispositivos móveis. A justificativa reside na estabilidade do sinal e no fato de o sistema de monitoramento ser uma instalação estacionária. Dessa forma, elimina-se a dependência de manter um smartphone fisicamente próximo ao hardware para garantir a transmissão.
- **Protocolo e Banco de Dados (Firebase):** O Firebase foi adotado como solução única de banco de dados para priorizar o MVP. Sua capacidade de atualização nativa por push permitiu que o foco do desenvolvimento fosse a estabilidade da comunicação direta. O uso de protocolos como o HTTPS (mesmo com desafios de firewall) foi mantido para garantir a segurança e a integridade do dado sem depender de terceiros.
- **Interface de Teste (Site Funcional):** Para não sobrecarregar as equipes de Frontend e Backend nesta fase inicial, foi desenvolvida uma interface funcional própria (index.html). Este site permitiu validar todo o fluxo de dados e o sincronismo em tempo real de forma independente, provando que o hardware está "conversando" corretamente com a nuvem.

6. Focos de Pesquisa Atual e Próximos Passos

Com o sucesso do **MVP**, a pesquisa agora entra em uma fase de refinamento técnico e robustez operacional. Os esforços estão divididos em frentes de infraestrutura, hardware e integração de software:

6.1. Infraestrutura e Autonomia de Hardware

- **Alimentação Independente:**

Está em fase de especificação pela equipe de hardware a aquisição de fontes de alimentação dedicadas para o ESP32. O objetivo é desvincular o protótipo da conexão USB do computador, permitindo que o sistema opere de forma autônoma e permanente no local de instalação.

- **Viabilidade de substituição de ferramentas:**

Iniciamos uma pesquisa para avaliar se as ferramentas são as mais eficiente para a escala final do projeto ou se há variantes (como o ESP32-S3 ou MQTT) que ofereçam melhor gerenciamento de energia e conectividade.

6.2. Resiliência de Dados e Conectividade

- **Transposição de Barreiras de Rede:**

Investigação de métodos para garantir que a comunicação HTTPS não seja interrompida por firewalls restritivos, assegurando que o dado chegue ao destino independente da rede onde o hardware for instalado.

- **Persistência Local (Buffer de Dados):**

Desenvolvimento de lógica para armazenamento temporário de dados no microcontrolador. Caso ocorra uma queda de internet, o dispositivo guardará o histórico de acionamentos da válvula e fará a sincronização automática assim que a conexão for restabelecida.

- **Estabilidade de Transmissão:**

Otimização do firmware para garantir que o sistema recupere conexões perdidas em milissegundos, mantendo a integridade da telemetria sob condições de sinal oscilante.

6.3. Evolução das Ferramentas Cloud e Web

Integração com o Aplicativo Definitivo:

A equipe de *Frontend/Backend* está em fase de definição da stack tecnológica, com forte tendência ao uso de React. O foco atual é garantir que a estrutura de dados que foi consolidada no Firebase seja facilmente utilizada por essa nova aplicação, permitindo uma transição fluida da interface de teste (index.html) para o aplicativo oficial.

7. Documentação da Implementação Técnica

(Firmware e Interface)

Para garantir a reprodutibilidade e a análise por outros técnicos, seguem os trechos fundamentais da lógica de comunicação direta desenvolvida para o ESP32 e a interface de validação.

7.1. Firmware de Comunicação (C++)

O código abaixo destaca a priorização do MVP, utilizando a biblioteca Firebase-ESP-Client para garantir o sincronismo nativo sem a necessidade de intermediários:

```
#include <Arduino.h>
#include <WiFi.h>
#include <Firebase_ESP_Client.h>
#include <addons/TokenHelper.h> // Auxiliar oficial da biblioteca
```

```
#define WIFI_SSID "NOME DO WIFI"
#define WIFI_PASSWORD "SENHA WIFI"
#define API_KEY "SUA API KEY"
#define DATABASE_URL "SUA DATA BASE URL"
```

```
FirebaseData fbdo;
FirebaseAuth auth;
FirebaseConfig config;
```

```
const int sensorPin = 15;
```

```

bool ultimoEstado = -1;

void setup() {
  Serial.begin(115200);
  delay(1000);
  pinMode(sensorPin, INPUT_PULLUP);

  Serial.println("\n--- Conectando ---");
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\nWi-Fi OK!");

  config.api_key = API_KEY;
  config.database_url = DATABASE_URL;

  if (Firebase.signUp(&config, &auth, "", "")) {
    Serial.println("Auth: OK");
  } else {
    Serial.print("Erro Auth: ");
    Serial.println(config.signer.signupError.message.c_str());
  }
  config.timeout.serverResponse = 10 * 1000; // 10 segundos

  Firebase.begin(&config, &auth);
  Firebase.reconnectWiFi(true);
}

void loop() {
  // Verifica se o Firebase está pronto (isso resolve o Token vazio)
  if (Firebase.ready()) {
    bool estadoAtual = digitalRead(sensorPin);

    if (estadoAtual != ultimoEstado) {
      int valorParaEnviar = (estadoAtual == LOW) ? 1 : 0;
      Serial.printf("Enviando Estado: %d\n", valorParaEnviar);

      if (Firebase.RTDB.setInt(&fbdo, "/valvula/estado", valorParaEnviar))
      {
        Serial.println("Sucesso no Firebase!");
        ultimoEstado = estadoAtual;
      } else {
        Serial.print("Erro no envio: ");
        Serial.println(fbdo.errorReason().c_str());
      }
      fbdo.clear(); // Limpa memória
    }
  }
  delay(200);
}

```

}

7.2. Interface de Monitoramento (Frontend Web)

```

<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>Monitoramento de Válvula - PI</title>
</head>
<body>

  <h1>Status da válvula:</h1>
  <h2 id="status">Carregando...</h2>

  <script type="module">
    import { initializeApp } from
    "https://www.gstatic.com/firebasejs/10.7.1/firebase-app.js";
    import { getDatabase, ref, onValue } from
    "https://www.gstatic.com/firebasejs/10.7.1/firebase-database.js";

    // --- Insira suas informações do Firebase aqui ---
    const firebaseConfig = {
      apiKey: "SUA_API_KEY",
      authDomain: "SEU_PROJETO.firebaseio.com",
      databaseURL: "https://SEU_PROJETO-default-rtdb.firebaseio.com",
      projectId: "SEU_PROJETO_ID"
    };

    const app = initializeApp(firebaseConfig);
    const db = getDatabase(app);
    const statusRef = ref(db, 'valvula/estado');

    // Sincronismo Push: Atualiza automaticamente ao detectar mudança
    no banco
    onValue(statusRef, (snapshot) => {
      const valor = snapshot.val();
      document.getElementById("status").innerText =
        valor == 1 ? "● Aberta" : "● Fechada";
    });
  </script>

</body>
</html>

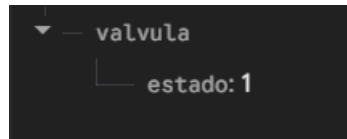
```

7.3. Estrutura de Dados (Firebase Realtime Database)

Caminho Principal (válvula/): Agrupa todos os sensores e atuadores relacionados ao sistema de reaproveitamento de água cinza.

Chave (estado): Armazena o valor booleano ou inteiro enviado pelo ESP32.

Valor 1: Indica que a válvula está em estado Aberto (acionamento detectado no Pino 15).



Valor 0: Indica que a válvula está em estado Fechado.

